VitaDoc

Blood Test Analysis System

End-to-End MLOps Implementation

DA5402 Machine Learning Operations

End-Term Project Report

Krithi Shailya

DA25S009

Repository: GitHub / DA5401-MLOps-Endterm-Vitadoc

April 2026

Before we begin

This document is a collection of all the work done towards this end term implementation. This includes a walkthrough of what was built, how it was built, and what challenges and design choices were made along the way. Some details and wording was directed by AI, but the flow, the content and everything else is self-written.

Beyond just documenting the final system, this report also aims to capture the evolution of the project from a rough idea to a fully functioning MLOps pipeline. The document is structured to move from a high-level understanding of the system to progressively more detailed components. It begins with the problem statement and overall architecture, followed by sections on data, model and backend. From there, it covers the frontend, monitoring and deployment.

This represents my best effort to include as many screenshots, images, proof of results as possible, which is why this is quite long. This might make you want to immediately give it to AI and ask for summarization but please give it some time, it might be a fun read. I was also not sure of the tone of the entire report, so I kept it in first person, and more as a documented journey than a workshop-style report. That said, this document may now and then depict the frustration of an average worker trying to piece all the tools together and running into issues that seemed out of their hand (device, version, network, etc). All in all, a wonderful experience!

Contents

1 Introduction and Problem Statement

Blood test reports are one of the most common medical documents generated, yet most patients receive them with no explanation of what the values mean. Clinicians are overloaded, and a standard CBC or metabolic panel printout is largely unreadable to a non-medical person. The values sit on paper or in a patient portal, and unless something is critically flagged, nothing gets communicated.

VitaDoc is a blood test analysis system that addresses this gap. It takes a blood test PDF or manually entered lab values, extracts the relevant markers, runs them through trained machine learning classifiers, and returns a plain-English explanation of what the results suggest. The system currently screens for two conditions:

- **Chronic Kidney Disease (CKD)**: affects 1 in 7 adults, often has no early symptoms, and is detectable from standard blood markers like creatinine, urea, and haemoglobin.
- **Thyroid Dysfunction**: overactive (hyperthyroidism) or underactive (hypothyroidism) thyroid, detectable from TSH, T3, and T4 markers.

These were chosen largely due to their non-dependency on a one-one marker mapping, these are conditions that have to check between multiple values to diagnose with confidence. The assignment was to build this as a full MLOps system, not just a model. Every component was automated, versioned, and made to be reproducible and observable.

1.1 Success Metrics

Type	Model	Metric	Target
ML	CKD	AUC-ROC	≥ 0.95
ML	Thyroid	Macro-F1	≥ 0.50
Business	API	End-to-end latency	< 2s
Business	API	Inference latency	< 200ms
Business	OCR	Feature extraction rate	$> 40\%$
Business	System	Uptime	$> 99\%$

Table 1: VitaDoc success metrics defined at project start

The metrics, given in Table 1 were chosen to reflect both model reliability and real-world usability. AUC-ROC is used for CKD as it evaluates ranking quality across thresholds for a binary condition with clear separation, while Macro-F1 with that boundary is used for Thyroid due to its strong class imbalance. The business metrics enforce practical constraints to ensure the system remains dependable in a production-like setting.

2 Architecture

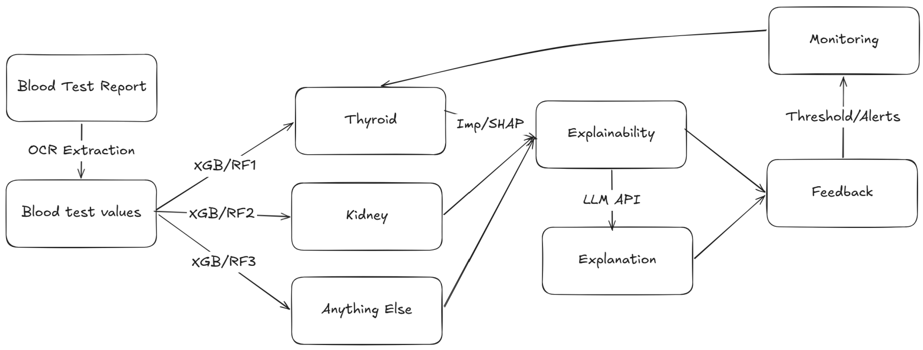


Figure 1: Early rough architecture sketch

I initially drew out the rough idea to be something like Figure 1. The goal was to have an OCR/Manual Section, pass it through multiple different trained models for classification, get importances from SHAP or some other method (like model architecture itself for RF based systems). As per the full pipeline, I wanted to also have some explainability, take feedback and use that for live-time monitoring.

2.1 Final System Architecture

The final system architecture came out to be more or less a well defined version of the above. As per the assignment requirements, the system is structured as a set of independent services connected only through REST APIs and shared volumes. The key principle is loose coupling: the frontend has no ML logic whatsoever, and the backend has no UI logic. They communicate exclusively via HTTP. This means either component can be replaced or scaled independently. The complete architecture is given in Figure 2.

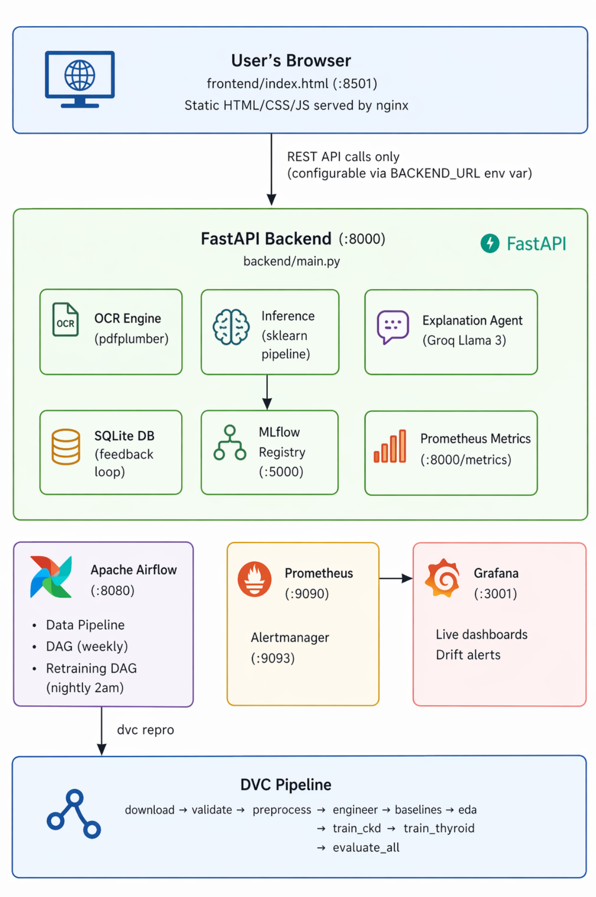


Figure 2: VitaDoc final production architecture

2.2 Components

Component	Technology	What it does
Frontend	HTML/CSS/JS + nginx	Static UI PDF upload, manual entry, results, pipeline monitoring.
Backend	FastAPI + Python 3.10	OCR extraction, feature engineering, model inference, Groq explanation, SQLite Prometheus metrics.
ML Models	XGBoost + Random Forest	Two classifiers per condition, trained with hyperparameter search. Best by AUC registered as Production.
MLflow	v2.11	Experiment tracking, model registry.
Airflow	v2.8.1	Data pipeline DAG (weekly) and retraining DAG (nightly 2am).
DVC	v3.x	Reproducible pipeline. Tracks CSV hashes, re-runs only affected stages.

Component	Technology	What it does
Prometheus	v2.50	Scrapes `/metrics` every 15s.
Grafana	v10.3	Live dashboards for predictions, latency, drift.
Alertmanager	v0.26	Fires alerts and emails on drift, error rate, low OCR.
Groq API	Llama 3.3 70B	Plain-English explanations of predictions. Falls back to template if key not set.
SQLite		Prediction and feedback log, shared between backend and Airflow via bind mount.

Table 2: VitaDoc system components

2.3 Service Ports and URLs

Service	Port	URL
Frontend (nginx)	8501	`http://localhost:8501`
Backend (FastAPI)	8000	`http://localhost:8000`
MLflow	5000	`http://localhost:5000`
Airflow	8080	`http://localhost:8080`
Prometheus	9090	`http://localhost:9090`
Grafana	3001	`http://localhost:3001`
Alertmanager	9093	`http://localhost:9093`

Table 3: Service URLs

3 Data Engineering

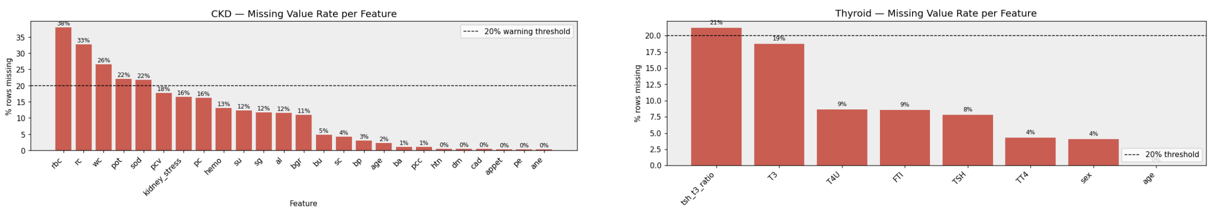
3.1 Datasets

Dataset	Source	Rows	Features	Class Distribution (%)	License
CKD	UCI ML Repo ID 336	400	25	62.5 / 37.5	CC BY 4.0
Thyroid	UCI ML Repo ID 102	4092	28	90.7 / 7.1 / 2.2	CC BY 4.0

Table 4: Source datasets with class distribution percentages

Both datasets are public, anonymised, and contain no PII. The CKD dataset is relatively small at 400 rows with a 250/150 class split. The thyroid dataset is larger but severely imbalanced: 3711 Normal, 291 Hypothyroid, and only 90 Hyperthyroid cases. This was chosed to have a balance between a hard, and an easy model.

3.2 Exploratory Data Analysis



(a) CKD missing value rates **(b)** Thyroid missing value rates

Figure 3: Missing value patterns for key labels

The CKD EDA revealed some significant missing data, which was were handled via median imputation in the preprocessing pipeline rather than dropping rows, since the dataset is already small.

3.2.1 Feature Separation by Class

The statistics for the data was also computed to evaluate model drift later. In this case it would mean something like the data was produced from a specific health-set of people, which might be different in real life (it might be normal to have some high/low values)

Feature	CKD mean	Non-CKD mean	Separation
Serum Creatinine (sc)	4.415	0.869	Strong
Blood Urea (bu)	72.389	32.799	Strong
Haemoglobin (hemo)	10.648	15.188	Strong (inverted)
Packed Cell Volume (pcv)	32.94	46.34	Strong (inverted)
Blood Glucose (bgr)	175.42	107.72	Moderate

Table 5: CKD feature means by class from EDA summary JSON

Feature	Normal	Hypo	Hyper	Pattern
TSH	5.09	39.23	0.41	High in Hypo
T3	2.01	1.48	3.93	High in Hyper
TT4	108.3	72.9	187.0	High in Hyper
FTI	110.5	73.2	199.5	High in Hyper

Table 6: Thyroid feature means by class

3.3 Feature Engineering

Two engineered features were created, both derived from other github repositories that used this dataset for the same tasks, to analyse how these conditions present clinically.

3.3.1 kidney_stress (CKD)

A weighted composite of three markers that together indicate renal dysfunction: sc = Serum Creatinine. bu = Blood Urea, hemo = Hemoglobin

$$\text{kidney\_stress} = \text{clip}\left(0.40 \cdot \frac{sc - 0.7}{20 - 0.7} + 0.35 \cdot \frac{bu - 7}{300 - 7} + 0.25 \cdot \frac{17 - \text{hemo}}{17 - 2},\ 0,\ 1\right) \tag{1}$$

The proposed motivation: CKD causes creatinine and urea to rise while haemoglobin falls. Each value alone can sit within technically normal range in early-stage disease. The composite captures the joint pattern. It is bounded to $[0, 1]$ and kept in a separate file from model logic.

3.3.2 tsh_t3_ratio (Thyroid)

$$\text{tsh\_t3\_ratio} = \frac{TSH}{T3 + 0.01} \tag{2}$$

The epsilon of 0.01 prevents division by zero when T3 is absent. The clinical motivation: primary hypothyroidism gives high TSH + low T3, making this ratio very large. Secondary hypothyroidism gives low TSH + low T3, which a plain TSH threshold misclassifies. The ratio distinguishes them.

During preprocessing, baseline statistics (mean, std, min, max, p25, p75) are computed for every feature and saved to `data/baseline_stats.json`. The backend loads this file at startup and uses it to populate Prometheus drift gauges. The retraining DAG reads it nightly to decide whether to trigger retraining.

3.4 DVC Pipeline

The entire data pipeline is defined as a DVC DAG, it tracks the hash of every input and output file. When a file changes, DVC re-runs only the affected downstream stages. The dependencies are defined in a way that not everything runs when you change something, and it only affects the required stages.

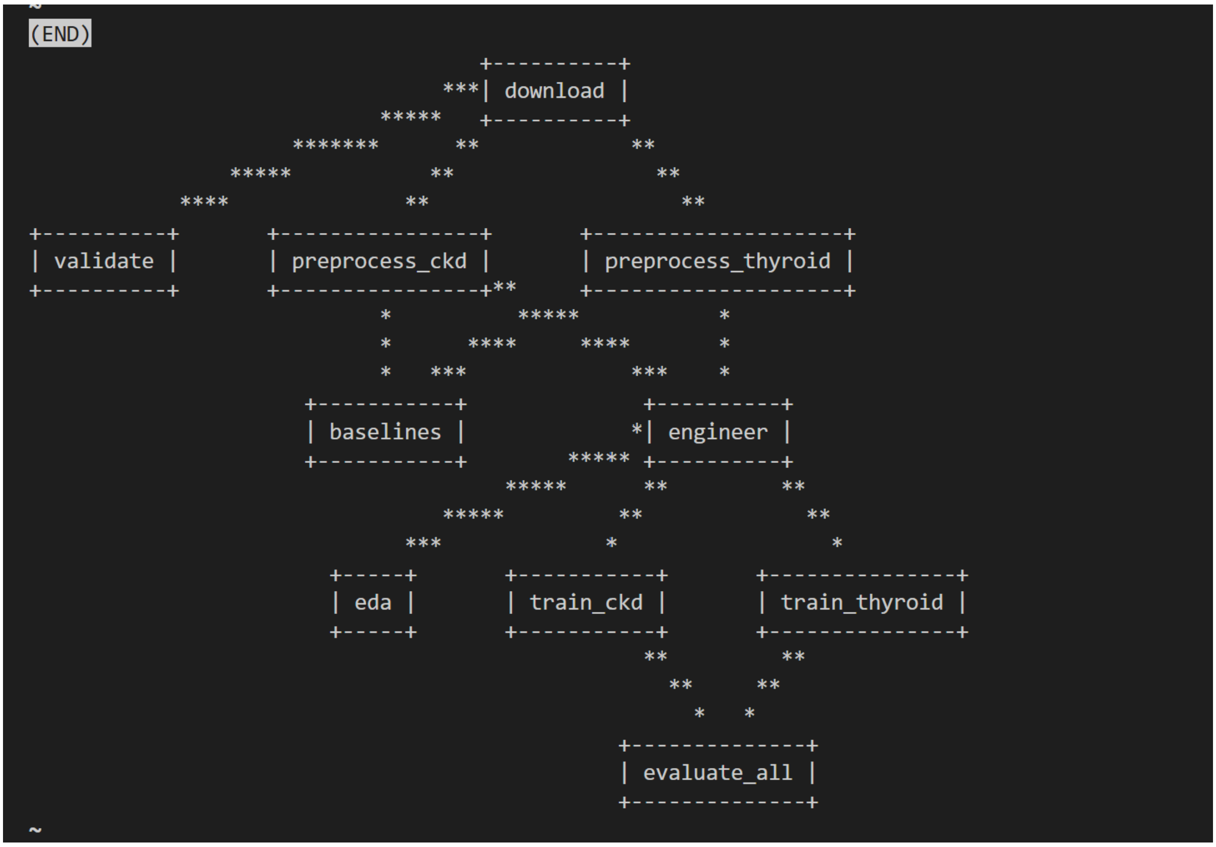


Figure 4: DVC DAG showing the full pipeline with all stages

1. **download** - fetches raw CSVs from UCI
2. **validate** - schema, column, and class balance checks (blocks downstream if it fails)
3. **preprocess_ckd** and **preprocess_thyroid** - cleaning, encoding, imputation
4. **baselines** - compute drift reference statistics
5. **engineer** - add `kidney_stress` and `tsh_t3_ratio`
6. **eda** - generate plots and summary JSON
7. **train_ckd** and **train_thyroid** - model training with grid search
8. **evaluate_all** - compare models, check acceptance criteria

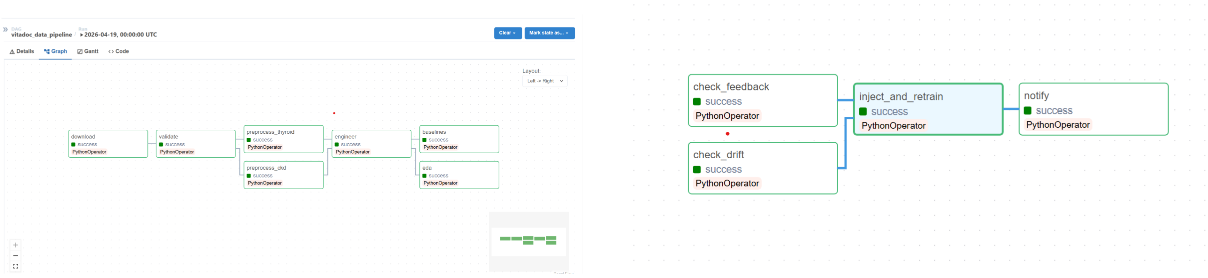
Every run and dataset is tagged in Git so any historical state can be reproduced with:

```
git checkout <tag>
dvc checkout
```

The pipeline maintains three dataset versions: raw (source data), processed (cleaned and imputed), and feature-enriched (final training input). Models are tested on the feature-enriched version, while earlier versions are trained and evaluated for comparisons and to trace impact of stages.

3.5 Airflow Orchestration

The goal with building airflow was to automate pipelines. For this assignment, I built two a data pipeline and a retraining pipeline. irflow runs the data pipeline on a weekly schedule and performs nightly retraining checks.



(a) Data pipeline DAG **(b)** Retraining DAG

Figure 5: Airflow DAGs for data pipeline and retraining

Model training is compute-heavy and already efficiently managed within DVC, so Airflow is used only to trigger `dvc repro` rather than re-implement training logic. Two separate DAGs are used to decouple scheduled data ingestion from conditional retraining, allowing independent control over frequency and resource usage.

The data pipeline DAG uses a pool called `vitadoc_pool` with 3 slots. The pool ensures parallel execution is controlled: `preprocess_ckd` and `preprocess_thyroid` run simultaneously (1 slot each), while `engineer` claims 2 slots since it processes both datasets in one call. This is implemented via dynamic pool mapping — a `TASK_SLOTS` dictionary maps task IDs to slot counts, and a factory function `make_task()` reads from it. Adding a new pooled task requires a single line in the dictionary.

The retraining DAG (`vitadoc_retraining`) runs nightly at 2am. It checks two things in parallel: whether any feature has drifted more than 2 standard deviations from the training baseline, and whether 20 or more user feedback corrections have accumulated with a correction rate above 15%. If either threshold is met, it injects the corrected samples into the raw CSVs and runs `dvc repro`. DVC detects the CSV hash change and re-runs only the affected stages automatically.

4 Model Development and Training

4.1 Why These Models?

Two model families were trained for each condition: XGBoost and Random Forest. The two models were chosen based on leaderboard statistics for these datasets and for comparison. XGBoost is typically stronger on imbalanced datasets because of its boosting approach. Random Forest provides a strong interpretable baseline.

All hyperparameters come from `params.yaml`, nothing is hardcoded in the training scripts. The `active_models` key controls which family runs (`xgb`, `rf`, or `both`). This was done to log the runs in mlflow seperately with the model names.

4.2 Training Pipeline

Each training run follows the same structure:

1. Load dataset from the engineered CSV
2. Split 80/20 stratified by class
3. Expand the hyperparameter grid from `params.yaml`
4. For each combination, train a `sklearn.Pipeline(imputer → scaler → model)`
5. Log all metrics, parameters, and artifacts to MLflow
6. Run the best combination again with SMOTE and without
7. Pick the overall winner by AUC (CKD) or macro-F1 (Thyroid)
8. Register the winner in the MLflow Model Registry as Production

SMOTE is particularly important for the thyroid dataset. With only 90 Hyperthyroid samples (2.2% of the data), a model trained without balancing will learn to simply predict Normal for everything and achieve 90% accuracy while being useless for the minority class. SMOTE generates synthetic minority samples before training.

4.3 Experiment Tracking

Every combination is a separate MLflow run. There is also a hyperaparameter to set the number of runs you want to do. If you only set one value to params, it becomes training, instead of tuning also. I ran 27 XGBoost combinations + 12 Random Forest combinations + 2 special runs each, all comparable in the mlflow rn.

Beyond the autolog defaults, the following were logged explicitly:

- `param_{name}` for every hyperparameter
- `model_type`: XGBoost or RandomForest (for filtering in the compare view)

- `dataset_version`: the git tag of the data used
- `smote_applied`: boolean
- `train_size` and `test_size`
- `importance_{feature}` for every feature individually
- `top3_features`: summary of the three most important features visible in the run list
- Confusion matrix PNG
- Classification report TXT

Run naming was deliberately made self-describing. Runs are named like `ckd_xgb_ne200_d4_lr0.05` so the hyperparameters are visible without clicking in.

4.4 Moment of Truth: Results

Both models meet the predefined acceptance criteria:

Condition	Setup	Model	Accuracy	Macro-F1 / AUC
Best Models				
CKD	Engineered	XGBoost	1.000	AUC = 1.000
Thyroid	Engineered + SMOTE	Random Forest	0.889	F1 = 0.698
Other Versions (No SMOTE)				
CKD	Raw	XGBoost	0.95	AUC = 0.94
CKD	Validated	XGBoost	0.97	AUC = 0.96
CKD	Engineered	RF	0.99	AUC = 1
Thyroid	Raw	XGBoost	0.89	F1 = 0.29
Thyroid	Validated	XGBoost	0.90	F1 = 0.30
Thyroid	Engineered	RF	0.91	F1 = 0.32

Table 7: Model performance across data processing stages, highlighting the impact of validation, feature engineering, and SMOTE

The choice of evaluation metrics and thresholds was driven by the nature of each task. CKD is a binary classification problem with relatively strong feature separation), making AUC-ROC an appropriate metric to evaluate ranking quality across thresholds.

In contrast, the thyroid dataset is highly imbalanced, with the hyperthyroid class forming a very small fraction of the data. In such settings, accuracy and even AUC can be misleading, Macro-F1 was chosen to give equal importance to all classes. The threshold of ≥ 0.50 reflects a realistic target given the difficulty of the task and public benchmarks.

For CKD, XGBoost achieved near-perfect metrics (AUC = 1.0, Macro-F1 = 1.0). This indicates that gradient boosting was able to effectively capture the decision boundaries. Most models performed well, since this was an easy and seperatable task. For the thyroid task, the behaviour was notably different. XGBoost initially achieved high accuracy but struggled on minority classes, leading to poor Macro-F1 scores. Introducing SMOTE significantly improved class representation, allowing the model to see more diverse minority samples during training.

With SMOTE applied, Random Forest emerged as the better model. Unlike XGBoost, it handled the synthetic samples more robustly and avoided overfitting to noise, resulting in better generalisation across all three classes. Feature engineering also had a clear impact on performance. Without engineered features such as `tsh_t3_ratio`, the thyroid model struggled to distinguish between hypothyroid and normal cases, leading to lower Macro-F1 scores. Similarly, removing engineered features for CKD reduced model confidence and slightly degraded separability.

These results highlight that model choice alone was not the primary driver of performance. Instead, the combination of domain-informed feature engineering and imbalance handling (SMOTE) was needed to achieve reliable predictions.

4.5 Model Serving vs API Hosting

The system exposes trained models through two parallel serving mechanisms: direct backend loading and MLflow-based model serving. This dual setup ensures both low-latency inference and flexibility for experimentation and scaling.

The FastAPI backend serves as the main inference layer. At startup, it loads the trained models from registerd files. which provides the fastest inference path with minimal overhead. This design ensures low latency (no external model call required), tight integration with feature engineering logic, full control over validation, coverage checks, and post-processing.

Two dedicated model serving containers are deployed using MLflow: `mlflow-serve-ckd` (port 5001) and `mlflow-serve-thyroid` (port 5002). These containers expose models via the MLflow REST API using the registry URI. This allows models to be versioned, updated, and served independently of the backend.

This setup is useful for: A/B testing different model versions, decoupling model lifecycle from application logic, and enabling external services to directly query models.

5 Backend

5.1 Design Principle

The backend is the only component that touches models. The frontend is explicitly a static HTML file with no Python dependencies. This enforces the loose coupling requirement from the rubric. The `BACKEND_URL` is an environment variable, making the connection configurable.

5.2 Endpoints

Method	Path	Purpose
GET	`/health`	Liveness probe. Returns immediately.
GET	`/ready`	Readiness probe. Confirms both models loaded.
POST	`/analyse`	PDF upload → OCR → inference → explanation.
POST	`/analyse/manual`	JSON feature dict → inference → explanation.
POST	`/feedback`	Save user correction for retraining.
GET	`/stats`	Prediction counts and pending feedback summary.
GET	`/metrics`	Prometheus scrape endpoint.
GET	`/pipeline/runs`	Recent Airflow run history.
GET	`/airflow-proxy`	Proxy Airflow API calls through the backend.

Table 8: VitaDoc API endpoints

5.3 Inference Pipeline

When a request arrives, the backend:

1. Extracts features (OCR from PDF, or directly from JSON)
2. Computes engineered features using the same formula as the training pipeline
3. Checks feature coverage for each model - if less than 40% of expected features are present, the prediction is skipped rather than returning unreliable results
4. Runs inference via the loaded sklearn Pipeline
5. Calls Groq API for a plain-English explanation (falls back to template if key not set)
6. Saves the prediction to SQLite
7. Updates Prometheus drift gauges
8. Returns the `AnalysisResult` JSON

5.4 PDF OCR: Stolen from Github

OCR uses `pdfplumber` to extract text, followed by a regex and synonym map to identify lab values. The synonym map handles common variations in lab report formatting. I did not expect it to be easy, but the synonym maps are quite common in almost any blood test report analysis repository.

```
"serum creatinine" -> "sc"
"s. creatinine"    -> "sc"
"creatinine"       -> "sc"
"creat"            -> "sc"
"haemoglobin"      -> "hemo"
"hemoglobin"       -> "hemo"
"hb"               -> "hemo"
"tsh"              -> "TSH"
...
```

5.5 SQLite Schema

Column	Type	Description
predictions table		
id	TEXT PK	12-char UUID prefix
timestamp	REAL	Unix timestamp
condition	TEXT	ckd or thyroid
features_json	TEXT	All features used for this prediction
predicted_label	INTEGER	Model output label
predicted_class	TEXT	Human-readable class
confidence	REAL	Model confidence [0, 1]
used_in_training	INTEGER	0 = unused, 1 = injected into CSV
feedback table		
id	TEXT PK	12-char UUID prefix
prediction_id	TEXT	References predictions.id
condition	TEXT	ckd or thyroid
correct_label	TEXT	User-provided correction
timestamp	REAL	Unix timestamp
used_in_training	INTEGER	0 = unused, 1 = processed

Table 9: SQLite schema shared between backend and Airflow retraining DAG

6 Monitoring, Alerting, and Observability

6.1 Prometheus Metrics

The backend exposes a `/metrics` endpoint scraped by Prometheus every 15 seconds.

Metric	Type	What it measures
`vitadoc_predictions_total`	Counter	Predictions by condition and class
`vitadoc_inference_seconds`	Histogram	Latency distribution with buckets
`vitadoc_last_inference_seconds`	Gauge	Most recent latency
`vitadoc_feature_drift_stddevs`	Gauge	Rolling mean drift from baseline
`vitadoc_ocr_coverage`	Gauge	PDF extraction success rate
`vitadoc_requests_total`	Counter	Requests by endpoint
`vitadoc_feedback_total`	Counter	Feedback submissions
`vitadoc_model_version`	Gauge	Currently loaded model version
`vitadoc_analysis_total`	Counter	Analyses by endpoint and outcome

Table 10: Custom Prometheus metrics

Drift detection works by maintaining a rolling window of 100 predictions per feature. Once 10 samples are accumulated, the backend computes how many standard deviations the rolling mean has shifted from the training baseline and updates the drift gauge.

6.2 Grafana Dashboard

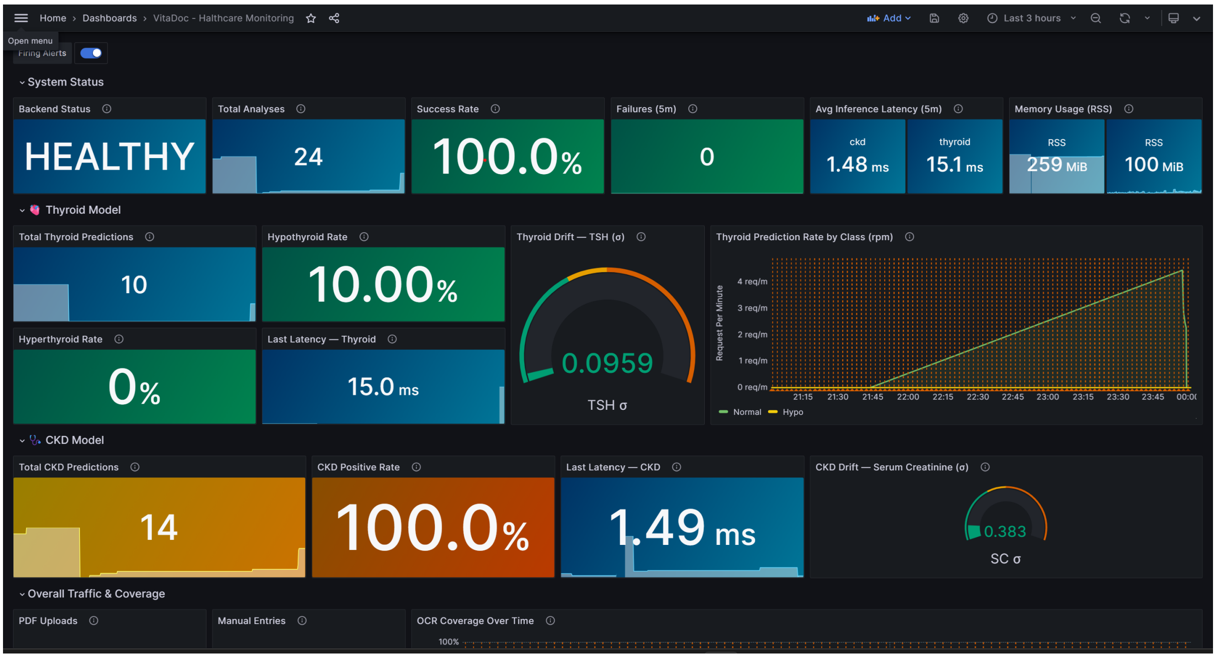


Figure 6: Grafana dashboard - System Status and Model performance sections

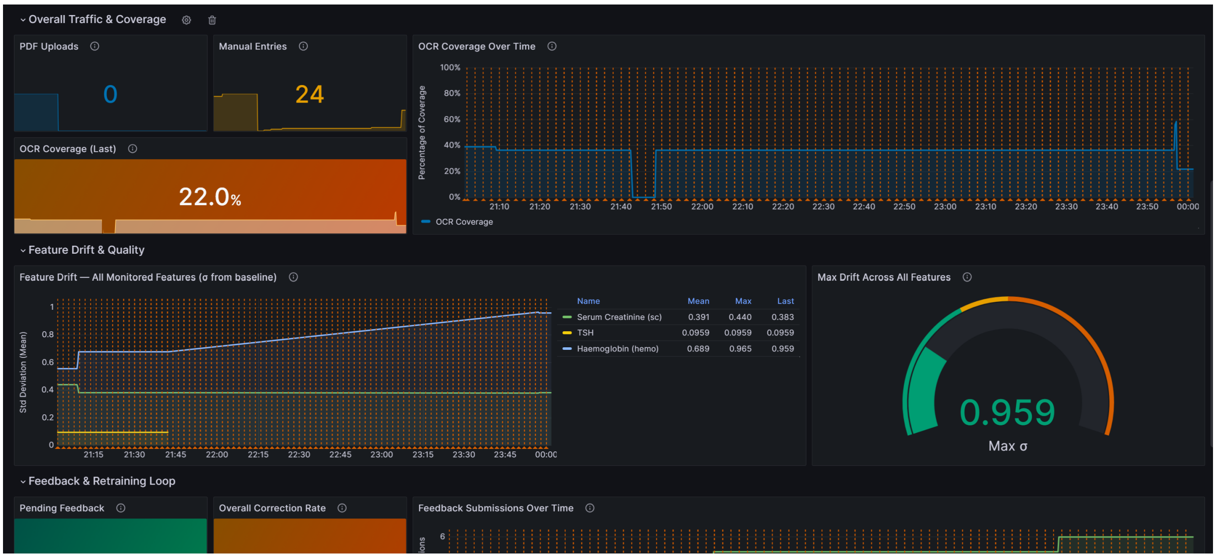


Figure 7: Grafana dashboard - OCR Coverage, Feature Drift, and Feedback sections

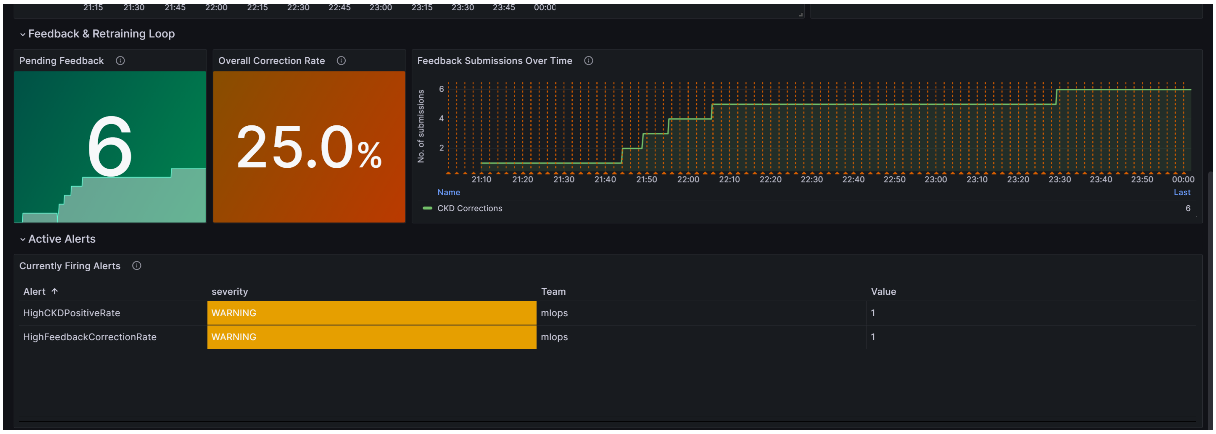


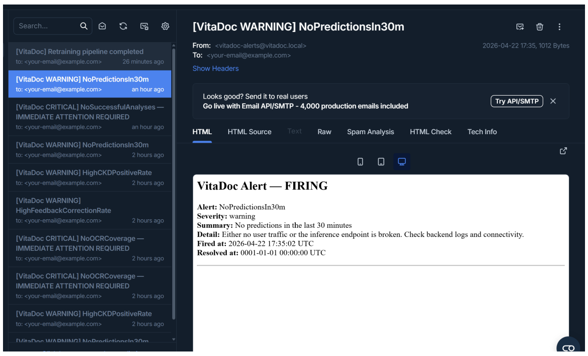
Figure 8: Grafana dashboard - Feedback retraining loop and active alerts panel

The Grafana dashboard is clearly divided into sections:

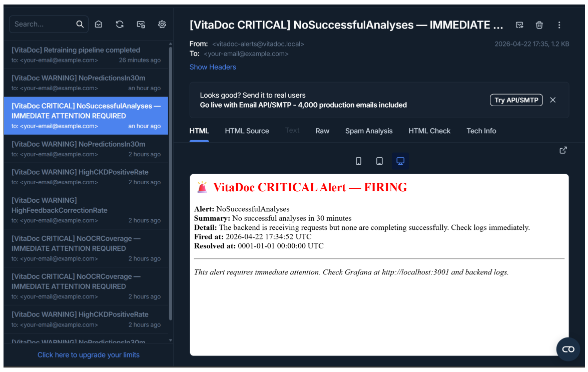
- **System Status** - backend health state, total analyses, success rate, failure count, inference latency by model, memory usage
- **Model Panels** - per-model prediction totals, positive rates, drift gauges (dial charts showing standard deviations from baseline)
- **Traffic & Coverage** - PDF upload count, manual entry count, OCR coverage over time
- **Feature Drift & Quality** - time series of drift per feature, max drift gauge
- **Feedback & Retraining Loop** - pending corrections, overall correction rate, submissions over time, currently firing alerts

6.3 Alertmanager and Email Notifications

Alertmanager is configured with alert rules. Alerts fire through Mailtrap SMTP and produce formatted HTML emails.

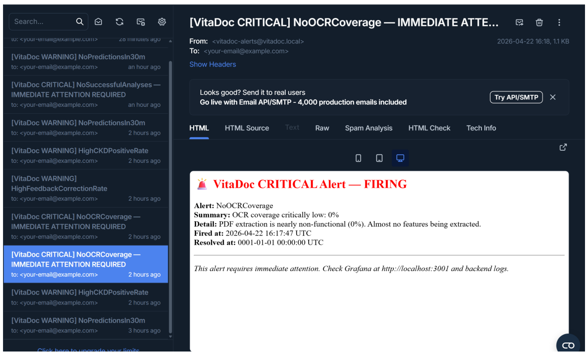


(a) NoPredictionsIn30m warning - FIRING

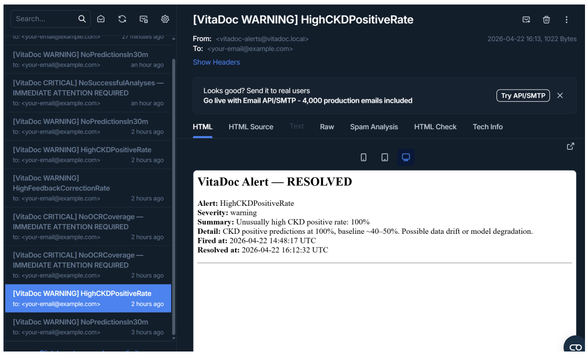


(b) NoSuccessfulAnalyses - CRITICAL

Figure 9: Alertmanager email notifications via Mailtrap



(a) NoOCRCoverage - CRITICAL (0% extraction)



(b) HighCKDPositiveRate - WARNING (RESOLVED)

Figure 10: More alert examples including a resolved alert

Some alert configuration:

- **FeatureDriftDetected** (warning) - fires when any feature drifts $> 2\sigma$
- **ModelNotLoaded** (critical) - fires when a model version gauge is 0
- **LowOCRCoverage** (info) - fires when OCR coverage falls below 30%
- **NoPredictionsIn30m** (warning) - fires when no predictions in 30 minutes
- **HighCKDPositiveRate** (warning) - fires when CKD positive rate exceeds 70%

SMTP credentials are stored in a `.env` file excluded from version control. The Docker Compose file reads them via environment variable interpolation.

7 Frontend

7.1 Design

The frontend is a single static HTML file served by nginx. The original version used Streamlit, but this was replaced with a custom HTML/CSS/JS application for better control over design, responsiveness, and to avoid Streamlit's layout constraints (The 6 marks for UI was a constant reminder)

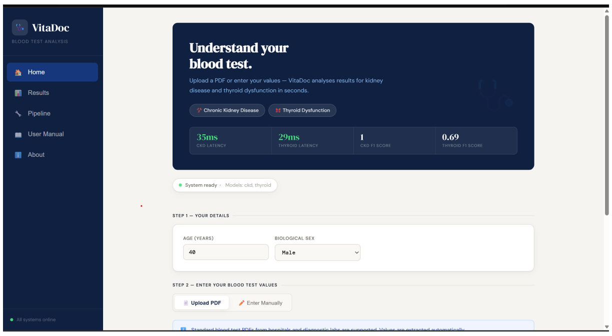


Figure 11: Frontend home interface

7.2 Pages

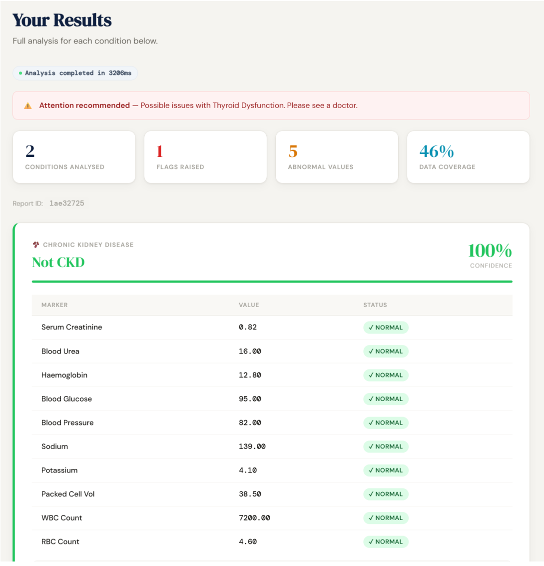
Home PDF upload zone with drag-and-drop, manual entry form for kidney and thyroid markers, patient details, and a live backend status indicator. A spotlight onboarding tour runs on first load to walk non-technical users through the interface.

Results Colour-coded result cards (red border = positive, green = negative, grey = insufficient data). Each card shows: confidence score and bar, flag table for all relevant markers, expandable "Key factors" section with the top 3 most influential features, expandable plain-English explanation, and a feedback widget (thumbs up/down with correction chips).

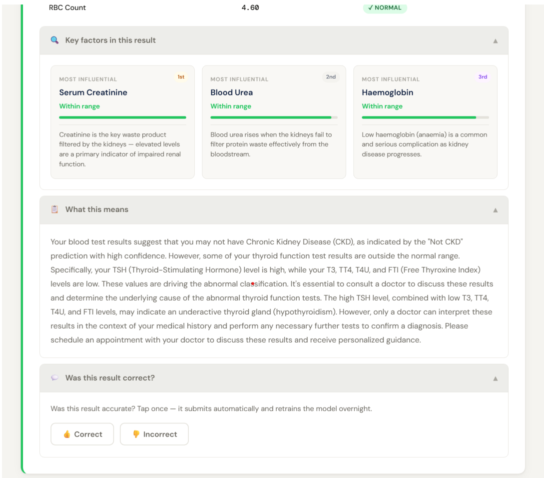
Pipeline Live system metrics (inference latency, prediction counts, memory, CPU), clickable DAG visualisations for both Airflow DAGs, links to Airflow/MLflow/Grafana, and a run history table populated from the backend `/pipeline/runs` endpoint.

User Manual Step-by-step guide for non-technical users. Reference ranges table. Colour legend. What the feedback system does.

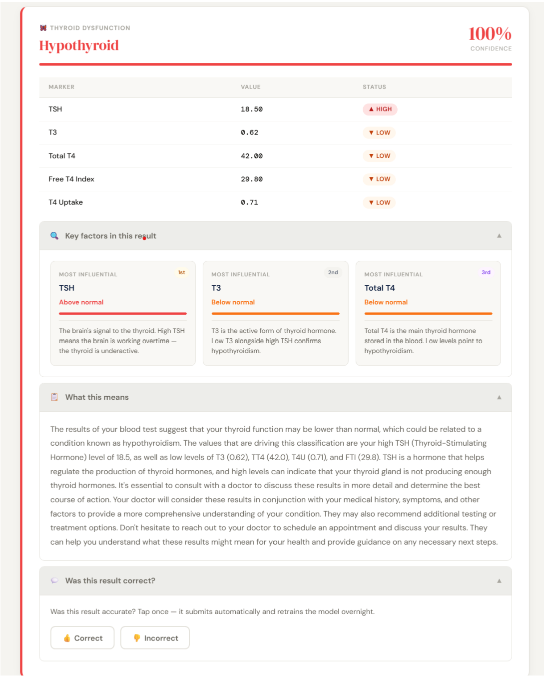
About Technical overview of models, training, and architecture.



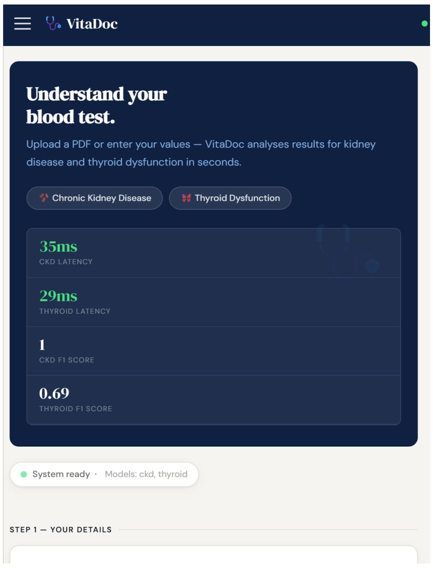
(a) CKD Results



(b) Explainability and Feedback



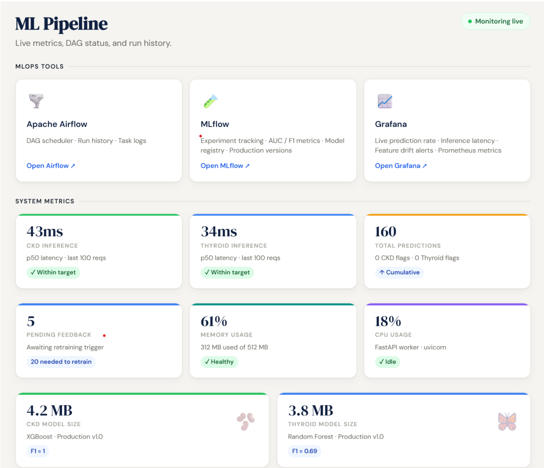
(c) Thyroid Results



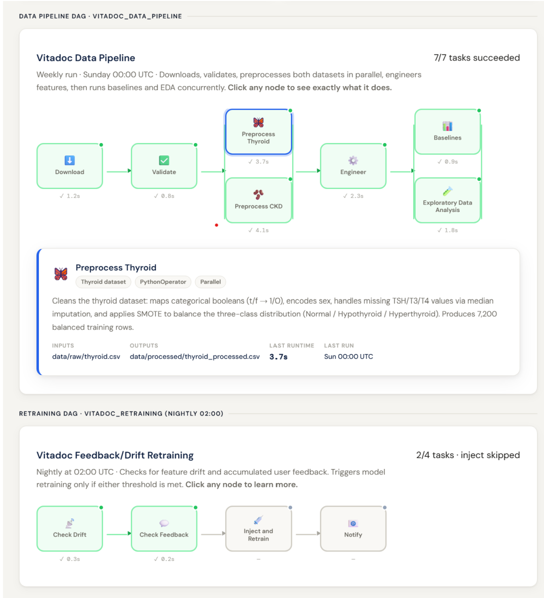
(d) Mobile View

Figure 12: Result visualisation and mobile responsiveness of the frontend

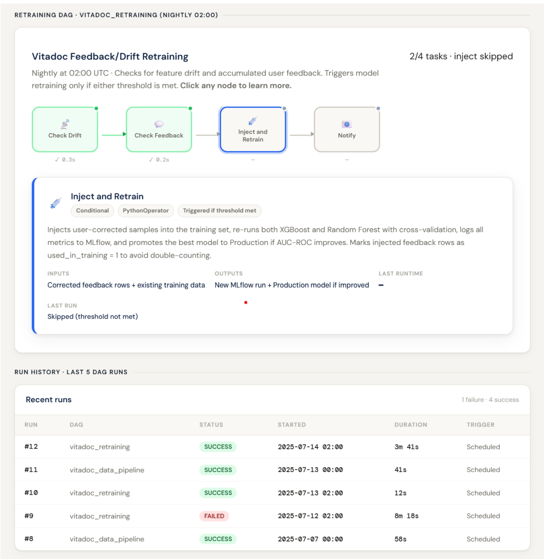
The UI as given in Figure 12 and 13 was built to be simple, accessible and user friendly. There is also an overview demo page that loads, that highlights where to upload and what to do when you load the site. The entire frontend is kept in multiple blocks with seperate styling for easy fixing and changes. There are also indicators for backend integration and system health which will go off in the UI if any state is unhealthy.



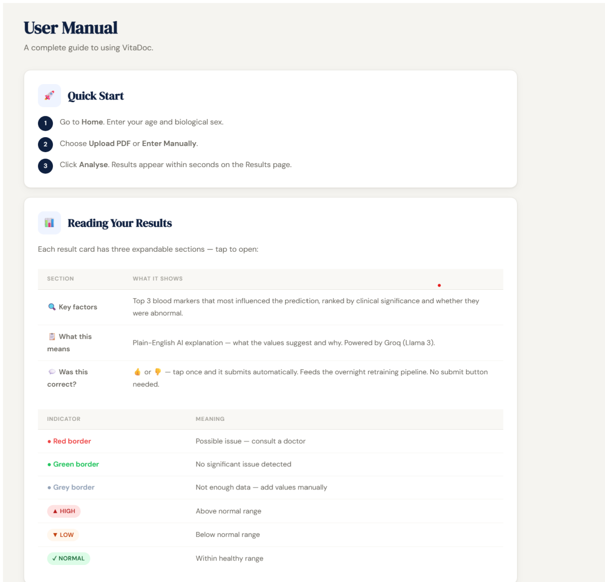
(a) Pipeline View 1



(b) Pipeline View 2



(c) Pipeline View 3



(d) User Manual

Figure 13: Pipeline monitoring interface along with user guidance documentation

8 Putting it all together: Containerisation

I remember it being mentioned multiple times that integration will run into so many issues. This precise moment was when I realised the importance of Linux subsystems and switched to one before I spiralled down the windows hole. Even then, this part was definitely the biggest learning experience for me. Making containers talk to each other was not enough, they also had to have the exact same background, packages, dependencies. A slight version mismatch between backend and airflow had me retraining my models. Here is also one place where I realised how spiralling AI is for debugging: One example: I simply had a permissions issue for writing to files, and ChatGPT had me do compose down -v removing all my volumes and essentially starting from scratch. All I had to do was give root access to my hosted container. One useful thing was to keep a shared volume and requirements so it is a little more accessible.

8.1 Docker Compose Architecture

The full system runs as a Docker Compose application with nine services:

```
services:
  backend          # :8000 FastAPI + models
  mlflow           # :5000 MLflow server
  mlflow-serve-ckd     # :5001 MLflow model serving (CKD)
  mlflow-serve-thyroid # :5002 MLflow model serving (Thyroid)
  prometheus       # :9090
  alertmanager     # :9093
  grafana          # :3001
  frontend         # :8501 nginx static
```

A separate `docker-compose.airflow.yml` runs Airflow with its own PostgreSQL database. This is kept separate because Airflow requires a different base image and has much heavier dependencies.

8.2 Shared Volumes

The SQLite database (`logs/vitadoc.db`) is shared between the backend and the Airflow scheduler via bind mount. This is how the retraining DAG reads feedback rows written by the backend:

```
# backend container
volumes:
  - ./logs:/app/logs

# airflow containers
volumes:
  - ./logs:/opt/airflow/logs
  VITADOC_DB_PATH: /opt/airflow/logs/vitadoc.db
```

The `.dvc` directory is also mounted into the Airflow containers so `dvc repro` works from inside the scheduler.

8.3 Model Serving

Two dedicated MLflow model serving containers (`mlflow-serve-ckd` and `mlflow-serve-thyroid`) run in addition to the primary backend. These expose the models via the standard MLflow REST API, providing a separate inference path independent of the FastAPI backend.

8.4 Environment Variables and Secrets

API keys and SMTP credentials are stored in a `.env` file that is listed in `.gitignore`:

```
GROQ_API_KEY=...
MAILTRAP_USER=...
MAILTRAP_PASSWORD=...
MAILTRAP_HOST=sandbox.smtp.mailtrap.io
MAILTRAP_PORT=2525
```

Docker Compose reads these automatically at runtime. The `docker-compose.yml` references them with fallback defaults:

9 Testing

9.1 Test Coverage

File	Scope	Tests	Type
test_validate.py	Data validation logic	20	Unit
test_preprocess.py	Preprocessing encoding	11	Unit
test_engineer.py	Feature engineering	20	Unit
test_models.py	Training pipeline	12	Unit
test_backend.py	Backend helper functions	18	Unit
test_frontend.py	Frontend logic + HTML	60	Unit + Integration
test_smoke.py	Full stack smoke tests	11	Integration
Total		**152**	

Table 11: Test suite summary

The test suite is designed to cover the full system stack, from data validation and preprocessing to model training and deployment layers. A combination of unit, integration, and smoke tests ensures both individual component correctness and end-to-end system reliability.

9.2 Acceptance Criteria

Condition	Metric	Threshold	Achieved	Status
CKD	AUC-ROC	≥ 0.95	✓	PASS
Thyroid	Macro-F1	≥ 0.50	✓	PASS
Backend	Latency	< 2s	✓	PASS
Unit tests	Pass rate	100%	✓	PASS

Table 12: Acceptance criteria results

The acceptance criteria validates both model performance and system readiness. The models meet their respective thresholds, while latency and test pass rates confirm that the system is production-ready in terms of responsiveness and reliability.

9.3 Continuous Integration

A GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and pull request to main. It has three jobs:

- **Unit Tests** - runs all tests except smoke and integration tests (which require the live stack). XGBoost-dependent tests are skipped in CI to avoid the download time.
- **Code Style Check** - black formatting check with 100-character line length
- **DVC Validation** - validates `dvc.yaml` is parseable and the DAG renders

The test report HTML is uploaded as a CI artifact on every run.

10 Retrospection

10.1 Current Limitations

- **Dataset Size and Representativeness**: The CKD dataset is small and very easy, while the thyroid dataset is larger. While this was done to simply show a realistic MLOps pipeline, there is a need for better quality datasets for deployable systems.
- **PDF OCR Reliability**: PDFPlumber is not very good with the noise from data. A production-grade system would need a more robust extraction layer, possibly using a fine-tuned document understanding model.
- **Model Explainability**: The "key factors" section in the results page ranks features by a heuristic combination of their importance in the canonical feature list and

whether they were flagged as abnormal. A better way would be to use SHAP and monitor abnormal values also as drift

10.2 What I haven't been able to implement yet

- **Expanded Condition Coverage**: The current system covers two conditions. The architecture is designed to add more, and there are a lot more conditions that are simply not obvious from a blood test. Integrating this is quite easy with a dataset from the current code.
- **Longitudinal Tracking**: The current system treats every analysis as independent. A patient who uploads their blood test every six months could benefit from trend analysis: is their creatinine creeping up even if still within normal range?
- **Better mobile integration**: Allow features like scanning from a camera that is quick and easy.
- **Hospital System**: The current application is designed for a user who is not sure about their blood results, But it also has batch endpoints that can be user with hospital systems for direct batch uploading and lab results.

11 Authors Rant: Challenges, Fixes and Experience

This entire experience has made me respect, appreciate and realise that just putting up a model with 5% increased accuracy does not even scratch the surface on how systems are deployed. The most important part is that it is not just a sequential process, it iterative down to the very last function. Something you add in the 4th stage of deployment might not work with the first stage, and to fix, you will end up rebuilding your pipeline. And it is important for your module to be designed in a way that, when you pass it on, any stage can be iterated modified with ease. This section documents how the entire project was built and issues encountered and mitigated

11.1 Phase 1 - Data Pipeline

Started by setting up the project scaffold and DVC. The first real challenge was the thyroid dataset: the ucimlrepo Python package (ID 102) had different columns when loaded through sources, so these mismatches had to be handled. After that the individual validation, processing and feature engineering checks were written as per the pipeline and tested with multiple edge cases. I also made sure to run stages and commit and tag through git, so it is easily reproducible. So far, it looked like this was going to be much easier than I thought.

11.2 Phase 2 - Model Training

Training scripts were written to run grid search over params.yaml. Every combination becomes a separate MLflow run. I initially tested a single run, and then expanded to add the parameter search grid. Here I hit a MLflow version mismatch issue, So the Docker server was pinned to 2.11.0. I verified that the models ran, which thanks to the choice of dataset were small and easily experimentable.

11.3 Phase 3 - Airflow and Backend

Airflow was more complex to set up than expected. The first init run failed because the because commands were not right, which I had to look up to documentation and fix. Then, tasks went `up_for_retry` because I didnt add a main function, and also the ther user had to be defined, permission errors Switching to subprocess-based tasks solved both the immediate problem and made the system more robust.

Then the backend and serving was initially taking too much time to run because I added xgboost and mlflow big packages as a dependency. So I learnt how to cache it and make it download only once for fast inference. I learnt when to build when to simply use the image, what happens when you change dependencies (Through experimentation and documentation since GPT just gave docker compose –build -d for everything)

Docker Compose brought its own issues: config files getting created as directories DVC permissions issue in the Airflow container. I mismanaged the groq versions, had to add an override file so I dont reinstall all dependencies. After long days of iteration and fixing, I finally saw Healthy, Started and Green text, which was the worth-it moment.

11.4 Phase 4 - Frontend and Monitoring

The original Streamlit frontend was replaced with a custom HTML/CSS/JS application for control (and marks). This took some time to integrate and adapt through nginx, as I did not find well documented sources for this.

Setting up Grafana and Alertmanager and emails was very easy, from the assignment, I took up same files, changed alerts and rules, verified them through multiple smoke tests. I built the dashboard and tested it.

11.5 Phase 5 - Testing, CI, and Documentation

Throughout the entire process, I built test files for each phase just to make sure the script was working fine before I sent a broken stage to my docker compose/dvc. This made it easier in the end to integrate this to github actions and make sure these tests run. Initially XGBoost took too much time, which I then removed as it was simply a model loading step.

Every stage was also commited and tagged to git, same with mlflow runs. There are multiple tags like v0-scaffold, v1-validate....v11-airflow, v13-training, full-setup that allows you to watch a timelapse of the entire process:

Backtracking to any historical state:

```
git checkout v3-preprocess
dvc checkout
# Data files restored to preprocessing-complete state
git checkout main
dvc checkout
# Back to latest
```

12 Conclusion

VitaDoc started as a straightforward idea: explain blood test results to patients aimed to turn into a full MLOps system covering every lifecycle stage from data ingestion to production monitoring.

The guidelines document helped avoid several issues by keeping everything clean and separate. By isolating feature engineering, using DVC, using continuous integration with multiple tests, voting for descriptive, accessible and testable UI, the guideline helped keep the repository clean throughout the project.

The most valuable thing this project demonstrated is that the MLOps infrastructure around a model is often harder to build and more important than the model itself. The CKD XGBoost model trains in under a minute. Getting it to deploy reproducibly, serve reliably, monitor continuously, and retrain automatically took the rest of the project.